

DASE7501-mp1

DASE7501-mp1 · 中文报告 · 2026-09-30
姓名：李昊澎 (Haopeng Li)
学号：3036803563

摘要

本项目在课程提供的 GPT 基线上，从随机初始化训练语言模型，逐步考察 dropout、SwiGLU、训练预算、宽度、深度以及旋转位置编码。初始基线验证 BPB 为 2.07109。在相同 1200 步训练预算下，近等参数 SwiGLU 将验证 BPB 降至 2.00788，但在宽度 192、6 层和 6000 步设置下，GELU + dropout 的 1.60688 优于 SwiGLU + dropout 的 1.63767，说明收益依赖实验设置。进一步采用宽度 256 的 RoPE、GELU 和多处 dropout 组合，得到 CPU FP32 验证 BPB 1.52770、完整测试 BPB 1.54400。该组合同时改变多项因素，不能把全部提升归因于 RoPE。本文报告有效和无效改动、消融对照及资源检查，并明确单种子与测试时间顺序的局限。

1. 任务、数据与基线

使用固定协议 7506-mp1-wt2-v2、课程 WikiText-2 数据和 BPE-2048 分词器。模型根据已观察前缀预测下一 token，评估窗口固定为 256 个目标，窗口之间不保留文本状态。验证集有 376,599 个评分目标、1,148,007 字节；测试集有 428,405 个评分目标、1,292,013 字节。所有模型保持词表、数据与评分协议一致。

BPB 定义为 $\Sigma[-\ln p(\text{真实下一 token} \mid \text{已观察前缀})] / [\ln(2) \times \text{原始 UTF-8 字节数}]$ ，越低越好。它不是准确率，也不能与采用不同分词协议的困惑度直接比较。

基线为 4 层、宽度 128、4 头 GPT，共 1,088,256 个参数，采用学习式绝对位置编码、Pre-LayerNorm、因果注意力、GELU 前馈网络、残差连接和词嵌入权重共享。项目复用课程代码，研究的是已知方法在本任务中的效果，不声称提出新的算法。

2. 最终模型设计

2.1 结构总览

组件	最终 RoPE 配置
Transformer 层数 / 隐藏宽度	6 / 256
注意力头数 / 每头维度	4 / 64
注意力类型	标准多头因果注意力；Q、K 全维度 RoPE
前馈网络	GELU，256 → 1024 → 256，含偏置
归一化	Pre-LayerNorm，输出前额外 LayerNorm
位置编码	固定频率 RoPE，base=10000；无学习式位置表
Dropout	0.10；embedding、注意力权重及两个残差分支
词嵌入 / 输出层	2048 × 256，权重共享
参数量	5,263,360
训练预算	6000 步，49,152,000 个训练目标

数据流为：token ID → embedding 与 dropout → 6 个因果 RoPE block → LayerNorm → 共享输出投影。每个 block 先执行 LayerNorm、Q/K 旋转、因果注意力与残差相加，再执行 LayerNorm、GELU 前馈层与残差相加。评估接口将 logits 转为 FP32 自然对数概率。

这里使用的是全维度 RoPE 和普通多头注意力；没有 GQA、QK normalization、可学习温度、输出门控、RMSNorm 或 EMA。它们出现在参考仓库的模型中，但不是本项目已完成的方法。

2.2 从基线到最终方案

前馈输出 **dropout**。在原前馈网络输出后、残差相加前加入 dropout，初期测试 0.10 和 0.05。它旨在抑制特征过度依赖，但是否有收益由验证结果决定，不能仅因其常用而假定有效。

SwiGLU。将原 Linear → GELU → Linear 替换为 $\text{down}(\text{SiLU}(\text{gate}(x)) \times \text{value}(x))$ ，使用输入相关的乘法门控。中间宽度取 $\text{round}(8d/3)$ ，以控制三个线性映射带来的参数增长。宽度 128 下仅比基线增加 168 个参数，因此早期比较近似匹配参数量。

训练预算及容量。分别考察 2400 与 6000 步、宽度 128 与 192、层数 4 与 6。相同训练目标数不等于相同计算量；同时，脚本的余弦学习率依赖总步数，改变步数也改变学习率轨迹。

RoPE 组合。最新候选移除学习式绝对位置表，在各注意力头的 Q、K 上按位置旋转，V 保持不变，继续使用因果掩码。正余弦由窗口内位置计算，每个窗口从零开始。该实现使用 GELU，并在输入 embedding、注意力权重、注意力输出和前馈输出上加入 dropout。当前候选还把宽度提高到 256、dropout 提高到 0.10。因此这是 RoPE 与容量、正则化的组合实验，不是纯粹的 RoPE 消融。实现继承课程 GPT 的通用接口，并参考了提供的 RoPE 示例代码；复用来源应在 README 中一并说明。

3. 实验设置

所有正式运行使用 seed=17、batch size=32、context=256。1200、2400、6000 步分别处理 9,830,400、19,660,800、49,152,000 个训练目标，重复采样也计入成本。训练器从随机初始化训练，没有 checkpoint 续训路径。

训练使用 AdamW，最大学习率 0.001，weight decay 0.1，100 步预热与脚本中的余弦衰减，梯度裁剪阈值 1.0。旧实验使用 CPU FP32；RoPE 训练日志记录 NVIDIA A100-SXM4-40GB、BF16，验证函数使用 FP32，另外完成了 CPU FP32 复现。不同设备和可能不同并发负载下的训练时间不能作为架构加速证据。日志未完整记录每次历史 CPU 型号和并发情况。

训练器保存最后一步而非验证最佳权重。全部主要实验为单种子，不声称统计显著性。下表来自现存 metrics.json；精确运行路径、数据和哈希可见配套实验表与 evidence JSON。

4. 主要结果与实验路径

4.1 最终结果摘要

指标	初始基线	最终 RoPE 组合
完整 CPU FP32 测试 BPB	2.10127	1.54400
验证 BPB (CPU FP32)	2.07109	1.52770
参数量	1,088,256	5,263,360
训练步数	1200	6000

最终测试 BPB 相比实际记录的初始基线降低约 **26.52%**。这是整套方案的改善：训练量增加五倍，模型容量、位置编码和正则化均有变化，不能将这一比例当作单独 RoPE 的贡献。最终测试分数仍高于原先希望达到的 1.5。

4.2 完整验证实验表

ID	Model	Width / depth	Dropout	Steps	Parameters	Val BPB	Train s	Device / precision
E1	GELU	128 / 4	0.00	1200	1,088,256	2.07109	209.56	CPU / FP32
E2	GELU	128 / 4	0.10	1200	1,088,256	2.08875	218.09	CPU / FP32
E3	GELU	128 / 4	0.05	1200	1,088,256	2.08191	232.16	CPU / FP32
E4	SwiGLU	128 / 4	0.00	1200	1,088,424	2.00788	187.87	CPU / FP32
E5	SwiGLU	128 / 4	0.05	2400	1,088,424	1.83138	474.00	CPU / FP32
E6	SwiGLU	128 / 4	0.05	6000	1,088,424	1.73074	1255.36	CPU / FP32
E7	SwiGLU	192 / 4	0.05	6000	2,223,232	1.66915	1644.08	CPU / FP32
E8	SwiGLU	192 / 6	0.05	6000	3,113,472	1.63767	2361.73	CPU / FP32
E9	SwiGLU	192 / 6	0.00	6000	3,113,472	1.66182	2411.34	CPU / FP32
E10	GELU	192 / 6	0.05	6000	3,111,936	1.60688	2611.32	CPU / FP32
E11	RoPE + GELU	256 / 6	0.10	6000	5,263,360	1.52770	158.95	A100 / BF16

4.3 训练动态

步数	6 层 SwiGLU + dropout , 宽度 192	6 层 GELU + dropout , 宽度 192
4000	1.66502	1.63785
4500	1.65261	1.62524
5000	1.64538	1.61609
5500	1.64046	1.60820
6000	1.63767	1.60688

两组后期验证成绩仍在下降，且 GELU 在这些记录点均优于 SwiGLU。RoPE 的 metrics.json 中 validation_history 为空，仅有最终验证结果，因此本报告不绘制或推断未记录的 RoPE 中间验证曲线。训练曲线的完整性与最终模型的可评分性是不同问题。

5. 消融：哪些改动有效，哪些没有稳定收益

对照	保持相同的条件	验证 BPB 变化	可以支持的结论
GELU → SwiGLU	128 宽度、4 层、1200 步、无 dropout	2.07109 → 2.00788	小规模短训练下门控有收益
SwiGLU → GELU	192 宽度、6 层、6000 步、dropout 0.05	1.63767 → 1.60688	较大设置下 GELU 更优
去掉 dropout	192 宽度、6 层、6000 步、SwiGLU	1.63767 → 1.66182	此设置下 dropout 有帮助
4 层 → 6 层	192 宽度、6000 步、SwiGLU + dropout	1.66915 → 1.63767	加深改善验证成绩，增加成本

早期结构对照（E1–E4）。相同 1200 步下，SwiGLU 相比 GELU 降低验证 BPB 0.06320；两种 dropout 比例单独加入基线时反而变差。说明短训练、小模型条件下，SwiGLU 有收益，而 dropout 未显示收益。固定种子并不保证各结构所有参数初始化完全一致。

训练量与容量（E5–E8）。同一组合从 2400 到 6000 步，BPB 从 1.83138 降至 1.73074。随后相同 6000 步下增宽至 192，降至 1.66915；再增加到 6 层，降至 1.63767。后两组分别隔离了特定设置下的宽度和深度变化，但计算成本随之增加。

六层消融（E8–E10）。去掉 SwiGLU、保留 GELU 和相同 dropout 后，BPB 从 1.63767 降至 1.60688，改善 0.03080；去掉 dropout、保留 SwiGLU 后，BPB 升至 1.66182，变差 0.02415。这说明 SwiGLU 并非在所有设置下更优，dropout 在这一 SwiGLU 设置中有帮助。该组相同训练量对照可作为关键机制的消融证据。纯 SwiGLU 在 5500 步达到 1.66141，但该步没有保存权重，不能把它当作现有 checkpoint 成绩。

RoPE 组合（E11）。CPU FP32 验证为 1.52769829，比 E10 的验证成绩更低，但它还改变宽度、dropout 的位置与比例、训练硬件和精度。缺少同宽度 256、同 dropout 设置、同训练条件下的绝对位置编码对照，因此不能隔离 RoPE 的独立贡献；现有 SwiGLU 消融也不能代替这一对照。若报告将 RoPE 作为唯一关键机制，这仍是待补的实验缺口。

6. 完整测试与资源检查

模型	完整 CPU FP32 测试 BPB	单次评分秒数
初始基线	2.10127	4.6794
宽度 128、2400 步 SwiGLU + dropout	1.86283	4.5065
宽度 256、6 层 RoPE 组合	1.54400	16.2610

最新测试精确值为 **1.543998614826954**，未达到 1.5 以下。各分数属于对应 checkpoint，不能互换。较早模型曾在后续实验前运行测试，因此不能声称全程开发均发生在首次测试之前；这一时间顺序应如实向教师说明，是否符合课程冻结要求由教师判断。

RoPE 与基线在同机 CPU、FP32、4 线程下，分别进行了三次独立进程的验证集评分。评分时间中位数分别为 10.808 秒和 4.250 秒，比例为 2.543，低于 5 倍上限。RoPE 最大进程峰值 RSS 为 1.735 GiB，低于 4 GiB；checkpoint 为 20.103 MiB，但尚未核算最终全部推理资产。内存包含加载与评分，评分时间不含加载。完整测试已有一次 16.261 秒记录，但没有同期基线重复计时及完整测试峰值内存，不能把验证集预检查写成完整测试资源认证。

原始小模型配置下 RoPE 的五项接口测试通过；对已训练模型，因果性、状态重置、梯度和单独复核的样本独立性通过。归一化残差约 1.62×10^{-3} ，超过单元测试 10^{-4} 容差，但低于评分器 10^{-3} 阈值。实际评分成功，报告不将此描述为所有严格测试均通过。

7. 成本与复现

现存记录覆盖 11 次正式训练和一次 10 步 smoke，共 353,976,320 个训练目标。CPU 训练累计 11607.37 秒，A100 训练 158.95 秒；分设备报告，不将其相加后称为统一计算成本。上述统计不包含未保存、失败或删除的运行、独立评分和人工开发时间。

数据学习仅使用提供的训练文本；开发记录采用验证集比较。实验代码保留原模型分支，RoPE 通过 `variant="rope"` 选择。当前 RoPE checkpoint SHA-256 为 49d0141168eb606d6be697087aa85aef6c3038d3a8b52a27fb6994fe726103f4，与现存权重及测试记录一致。按课程 README 安装依赖后，从 `code/` 运行；评分无需重新训练：

```
# Run from code/ after installing the documented dependencies.
python train.py --implementation student --config
    configs/rope_width256_depth6.json --device cuda --precision bf16 --threads
    4 --seed 17 --steps 6000 --eval-every 500 --run-dir runs/reproduce-rope
python evaluate.py --checkpoint runs/rope-width256-depth6-6000/checkpoint.pt
    --device cpu --precision fp32 --threads 4 --split test
```

训练命令重述记录中的主要设置，不能保证跨设备逐位一致。最终仓库需提供不可变代码链接、匹配的权重链接及明确路径，并确认配置、源码与评分文件对应。

8. 讨论与结论

本项目最重要的发现是改进效果依赖模型规模和训练设置：小模型的 SwiGLU 收益没有在较大模型中保持，dropout 的效果也随设置变化。容量扩展与更长训练改善了验证表现，但增加成本；RoPE 组合获得当前最低验证分数及 1.54400 的完整测试 BPB，其独立机制收益尚未分离。后续工作应优先补同条件 RoPE 消融、多种子稳定性及完整测试资源验证，而不是仅增加结构复杂度。

9. 个人工作与 AI 协助

个人工作。我阅读并逐步理解了课程 GPT 的模型和训练流程，在 AI 建议的基础上编辑 `student.py` 和配置文件，执行多轮训练，核对日志并比较不同设置。我主动提出增加 block 数量，讨论在继承 GPT 的基础上接入 RoPE，并根据实验需要补充 GELU、SwiGLU 与 dropout 对照。项目中的方案由我逐步选择和实施，而不是直接采用一份现成结果。对实验结论，我需要依据实际指标作判断，包括接受 GELU 在较大设置下优于 SwiGLU 的结果，并区分验证成绩、测试成绩和资源成本。

AI 协助范围。OpenAI Codex/ChatGPT 用于解释概念和规则、建议实验、提供部分实现示例、诊断错误、将提供的 RoPE 示例合入 `student.py`、运行部分接口及资源检查和一次早期测试评分，以及整理日志和起草双语报告。模型基线来自课程，RoPE 还参考提供的示例文件。这些帮助不替代我的实验执行、理解和最终责任；我需核对报告表述与个人实际工作一致，并在 README 中保留实质性 AI 帮助和代码复用说明。

10. 数据与报告组织说明

数据沿用课程提供的 WikiText-2，原始文本来自 Wikipedia，分词器仅由训练集拟合。再分发时保留课程 README 中的 CC BY-SA 3.0 与 GNU Free Documentation License 数据声明。正文未附独立参考文献列表。

本次修订参考了 vectorBH6/HKU_DASE7506_MP1 报告的展示层次（模型配置、主要结果、训练动态、消融及复现），未采用其模型组件、实验数字或个人贡献声明。所有数值来自本项目日志。结构参考：https://github.com/vectorBH6/HKU_DASE7506_MP1/blob/master/report/report.md